

STM32 FreeRTOS Room Multisensor

Laboratory Activity 1 - Academic Report

Kevin Christian Villareal

BCA182 | Repository: uninhabitableelbow/bca182-freertos-multisensor

Report date: 08 October 2026

1. Problem and Requirements

A room monitor must acquire several environmental signals while remaining responsive to navigation, motion, and alarm conditions. A single loop with long blocking sensor or display operations can delay unrelated work. This implementation decomposes the application into native FreeRTOS tasks with explicit ownership and communication.

Requirement	Implemented behavior
Measurements	DHT22 temperature/humidity; ADC relative light; PIR motion
Presentation	Encoder selects Temperature, Humidity, Light, and Motion OLED pages
Alarm	Buzzer below 18 C or above 30 C; 18 C and 30 C are normal
Activity	15 seconds without PIR motion enters INACTIVE; PIR wakes system
RTOS mechanisms	Bounded FIFO, latest-value queues, event group, mutexes, blocking delays
Quality evidence	34 executed pure unit tests, interpreted static findings, Wokwi functional tests and faults

Wokwi adaptation and scope

The simulator configuration uses an 8 MHz HSI clock and a custom cooperative Thread-mode FreeRTOS port with a 100 Hz kernel tick derived from TIM3. This adaptation followed the observed NVIC/port assertions and scheduling problems during simulation. Higher-priority ready tasks are selected at blocking or yield points; a running task is not interrupted by scheduler preemption.

A separate physical configuration retains the standard preemptive Cortex-M3 port, 72 MHz HSE/PLL, and 1 kHz tick. STM32Cube HAL and the bundled native FreeRTOS kernel are used without Arduino or CMSIS-RTOS wrappers. Simulator responsiveness does not establish physical timing, electrical correctness, or hardware preemption.

In this report, I document the implementation, host-test results, static analysis, and my observations in Wokwi. I distinguish functional checks from measurements of physical timing and electrical behavior, which I have not performed.

2. System Architecture and Design

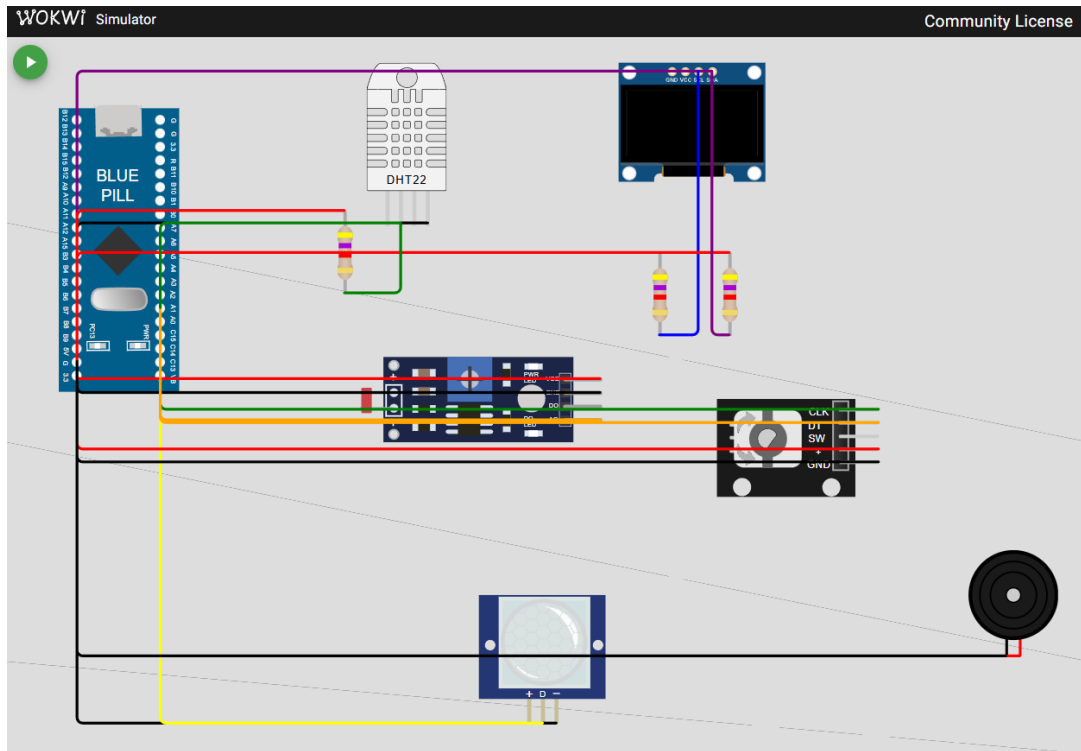


Figure 1. My Wokwi circuit screenshot: all sensing, input, display, and alarm components share the Blue Pill and common ground.

Subsystem	Interface	Owner
DHT22 / light	PA1 GPIO / PA0 ADC1; 3.3 V	SensorTask; TIM2 pulse clock
Rotary encoder	PA2 CLK, PA3 DT; pull-ups	InputTask; driver polling/EXTI
PIR	PA4 OUT; 5 V module in Wokwi	MotionTask
OLED	PB6/PB7 I2C1; 0x3c, 100 kHz	DisplayTask
Buzzer	PB8 TIM4 channel 3; 2 kHz PWM	AlarmTask
Diagnostics	PA9/PA10 USART1; 115200, 8N1	Task writers via serial mutex

Task modules orchestrate separate low-level drivers. SensorTask publishes copied messages to logging, display, and alarm consumers; InputTask sends the page selection. MotionTask publishes coherent activity snapshots and persistent flags.

ACTIVE permits sensing, navigation, display, and valid-sample alarm decisions. INACTIVE pauses sensing, turns off the OLED, ignores navigation changes, and silences the alarm while keeping PIR monitoring. PIR high refreshes the last-motion time; 15 seconds after it clears enters INACTIVE. Motion returns to ACTIVE. This is application inactivity, not a CPU low-power sleep instruction.

3. FreeRTOS Architecture

Task	Responsibility / trigger	Priority	IPC	Typical blocked condition
SensorTask	Measurements / 2 s	3	FIFO + two mailboxes; ACTIVE	Delay, DHT readiness, or ACTIVE wait
DisplayTask	OLED / 20 ms update check	1	Sample/mode mailboxes; events	Delay or ACTIVE wait when off
InputTask	Navigation / 10 ms	2	Mode mailbox; state snapshot	Periodic delay
MotionTask	PIR and activity / 50 ms	2	Event group; state mutex	Periodic delay
AlarmTask	Temperature decision / sample	2	Sample mailbox; ALARM flag	Queue receive up to 100 ms
SensorLogTask	Format sensor diagnostics	1	4-item sensor FIFO; serial mutex	Queue receive indefinitely
Task A / B	Distinct diagnostics / 1 s	1	Serial mutex; B observes events	vTaskDelayUntil

Scheduling justification

The handout's table suggests Sensor/Alarm at 2, Input/Motion at 3, and Display at 1. This implementation deliberately uses Sensor at 3 and Input/Motion at 2. DHT22 pulse measurements last tens of microseconds; task-level preemption can corrupt a synchronous read. Human navigation and long PIR pulses can tolerate the short bounded acquisition. Display and logging tolerate greater latency and stay at 1.

A task is Running while executing, Ready when runnable but waiting for CPU, and Blocked during a delay, queue/event wait, or contended mutex. The cooperative port preserves these states but cannot demonstrate interrupt-driven preemption. The alarm's 100-ms receive timeout permits periodic inactivity/staleness checks; the two-second producer period dominates temperature detection latency.

Communication and synchronization

A four-item FIFO preserves logging order; a full FIFO increments drops rather than stalling acquisition. Independent one-item mailboxes overwrite the latest display and alarm samples. A separate mailbox carries display mode. Messages include validity, sequence/drop counts, raw ADC, and activity epoch, so old samples cannot cross a sleep/wake transition unnoticed.

MotionTask owns ACTIVE/MOTION event levels; AlarmTask owns ALARM. Sensor/display waiters do not clear ACTIVE. The state mutex protects snapshots and transitions. The serial mutex protects the shared UART handle and complete diagnostic reports. Formatting occurs before taking it; state and serial locks are not nested. Native mutex priority inheritance helps a waiting higher-priority task's owner finish.

4. Implementation

Acquisition and validity

The DHT22 driver releases PA1 into input mode after its start pulse, reads a bounded 40-bit response using TIM2, and validates checksum and range before publishing. ADC1 reads the LDR analog output. Brightness is the rounded inverse of the 12-bit ADC scale; it is relative percent, not calibrated lux. Failed reads retain explicit validity rather than becoming believable zero measurements.

Periodic RTOS deadlines can occur slightly before two actual HAL seconds have elapsed since a previous DHT attempt. SensorTask therefore blocks for the remaining driver interval and rechecks activity/epoch before reading. This addressed observed Error/alarm cycling after PIR wake-up at 34 C without concealing genuine read errors.

Presentation, navigation, and alarm

DisplayTask owns I2C1, retries bounded failures, and caches OLED pages so unchanged pixels need not be sent. Changed spans are transmitted in small chunks. Shorter values erase earlier digits. AlarmTask accepts only fresh, current-epoch valid temperature samples; invalid, missing, or inactive states silence the buzzer. TIM4 generates the waveform without a CPU tone loop.

Wokwi encoder sampling runs at safe HAL/timekeeping and DHT loop points; the quadrature decoder counts complete cycles and rejects invalid transitions. InputTask translates position changes into wrapped page selections. Physical firmware uses a bounded falling-edge EXTI path. Simulator interrupts had caused lag during rapid turns, so the verified simulation avoids that interrupt storm.

Startup, module boundaries, and failure handling

main.cpp initializes HAL and clocks; app.cpp initializes drivers, creates checked RTOS objects/tasks, and starts the scheduler. sensors, display, input, alarm, motion, state, objects, and diagnostics are separate modules. Pure decision logic remains hardware-independent for tests. Allocation, UART, assertions, and stack overflow failures use bounded fault output and stop execution; fatal paths cannot take a task mutex safely. OLED error prefix/detail lines share one serial lock.

Normal mode 0 is selected in lab_fault_config.h. Fault modes isolate one deliberate change, identify it in the startup banner, and are blocked from physical builds. The selector does not build or simulate; I build the firmware again after each selection.

5. Verification and Testing

Executed pure-logic unit tests

Suite	Required	Passed	Coverage
Alarm	5	11	18/30 C boundaries, normal/outside values, invalid gating, NaN/infinity
Navigation	4	13	Forward/reverse transitions and wrap; complete cycles, bounce, rapid cycles
System state	4	10	Before/at timeout, inactive/motion, refreshed deadline, startup, timer wrap
Total	13	34	Production functions executed by Unity through pio test -e native

The host suites compiled with warnings treated as errors and executed successfully. They test actual alarm.h, navigation.h, and system_state.h functions, not copied models. Their scope is deterministic logic, not peripheral timing or concurrency.

Wokwi functional verification

ID	Stimulus	Expected / observed behavior	Result
FT-01	Change temperature	Temperature display updates	PASS
FT-02	Change humidity	Humidity display updates	PASS
FT-03	Dark/bright light	Relative light changes	PASS
FT-04	Clockwise rotation	Next page, including wrap	PASS
FT-05	Counterclockwise rotation	Previous page, including wrap	PASS
FT-06	34 C while ACTIVE	Alarm activates	PASS
FT-07	Return to 25.4 C	Alarm stops	PASS
FT-08	PIR while ACTIVE	Detected motion; stays ACTIVE	PASS
FT-09	15 s after motion clear	INACTIVE; OLED off	PASS
FT-10	PIR while INACTIVE	ACTIVE; display/readings resume	PASS

I followed the ten-test flow and verified that all listed functions worked. These PASS entries record my observations in Wokwi. I did not record separate timing measurements or screenshots for each test. The later screenshot shows one temperature display observation and does not independently document all ten tests.

5. Verification and Testing - continued

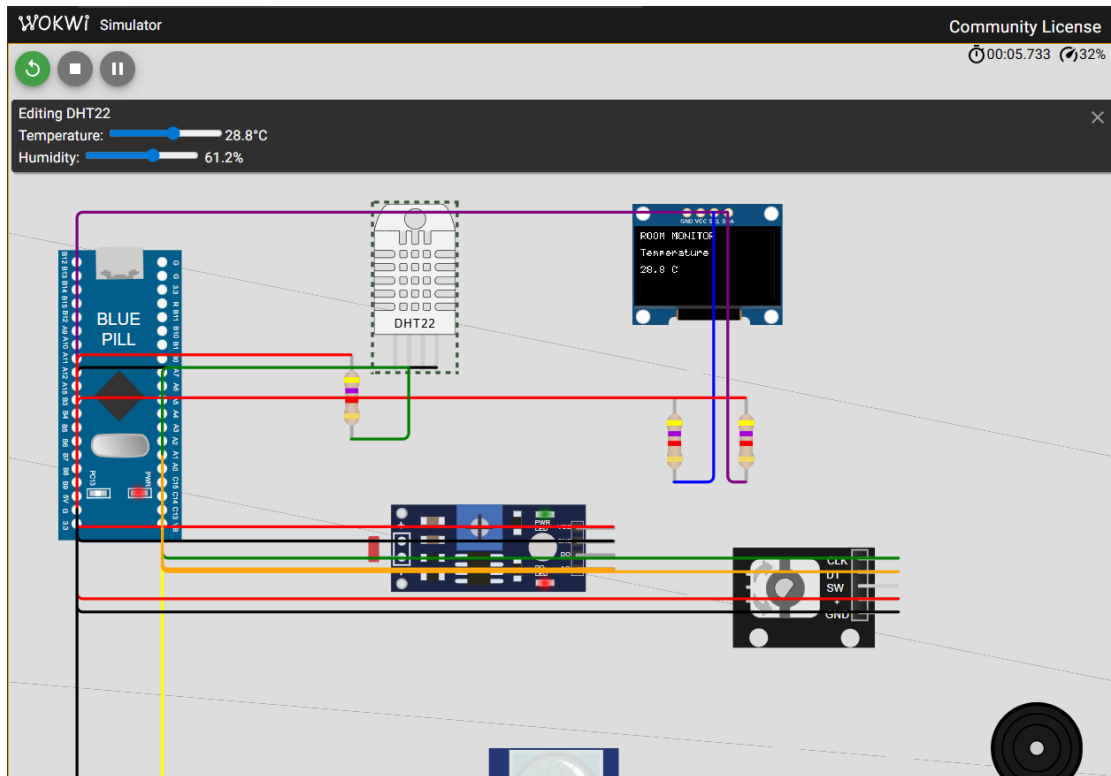


Figure 2. My simulation screenshot at 5.733 simulated seconds. The DHT22 control and OLED both show 28.8 C; the humidity control is 61.2 percent. The terminal is not visible, so no simultaneous UART measurement is claimed.

Deliberate FreeRTOS fault experiments

Fault	My observation	Technical interpretation
Remove Task A delay	22 consecutive Task A running lines; normal operation returned after restoration	Continuous printing is established. Starvation, clock progress, and CPU utilization were not measured.
Input priority 2 to 4	No visible problem observed	Cooperative selection changes at block points; InputTask still delays 10 ms. No visible degradation is plausible.
Remove serial mutex	Clean output; normal mode restored	Non-yielding UART calls can hide contention. Clean cooperative output does not prove preemptive safety.

No-delay work can monopolize a cooperative scheduler because ready higher-priority tasks cannot interrupt it. Under preemption, higher priorities can still run while lower tasks/Idle risk starvation. Excessive input priority can interrupt DHT pulse acquisition on hardware. Unprotected writers can encounter a busy HAL handle or interleave output depending on the path. These are explanations, not invented fault outcomes. All fault selections were compiler-checked; I confirmed normal operation after restoring mode 0, also verified in the source header.

6. Static Code Analysis

PlatformIO Cppcheck 2.11 analyzed both firmware configurations with C++11, a 32-bit pointer/int/long data model, and framework include paths. Initial pio check reported 0 high, 0 medium, and 34 low findings. After the OLED lookup correction, both configurations passed with 0 high, 0 medium, and 33 reviewed low findings. High/medium findings were configured to fail the final commands.

Finding / count	File / line examples	Cause and resolution
useStlAlgorithm / 1 resolved	oled.cpp:74 (initial)	Manual first-match font loop replaced with std::find_if; same unknown-character blank output. Absent in final checks.
unusedFunction / 24 low	alarm.cpp:9; sensors.cpp:12,76; system_state.cpp:5	PlatformIO invokes Cppcheck per source file; callers are in other modules or xTaskCreate. Retain functions; caller evidence documented.
cstyleCast / 8 low	buzzer.cpp:28; dht22.cpp:56; diagnostics.cpp:26,55	CMSIS register mappings and FreeRTOS xSemaphoreGive expand vendor C casts. Retain required interfaces, not blanket suppression.
constParameterPointer / 1 low	serial.cpp:10	HAL_UART_MspInit must match the mutable handle pointer in HAL declaration; retain signature.

Interpreting a passing result

Passing analysis does not mean zero findings or proof of runtime correctness. The complete 33-row file/line/cause/resolution table is retained in docs/part-xv-static-analysis.md. All retained functions have actual production callers. Deleting them based on per-file reports would break the application. No new suppressions hid these diagnostics.

The corrected OLED source compiled with strict ARM C++11 checks in both configurations, and all 34 unit cases still passed after correction. The analysis report is src-focused: framework source and the custom C port are not covered by that report. New findings must be inspected rather than compared only by count.

Reproduction commands

```
pio test -e native
pio check --fail-on-defect high --fail-on-defect medium
pio check -e bluepill_f103c8 --fail-on-defect high --fail-on-defect medium
```

I build the simulation firmware with `pio run -e bluepill_wokwi`, then start Wokwi in VS Code. I repeat the build after source changes so the simulator loads the current firmware.

7. Engineering Discussion

Problem observed	Investigation / decision	Evidence and remaining limit
FreeRTOS assertion and stopped prints	NVIC/port investigation led to cooperative simulation port; separate native hardware build retained	Recurring tasks later verified in simulation; simulator does not prove preemptive timing
DHT timeouts	Bounded start/response timing, explicit bus release, checksum/range validation	Changing temperature/humidity readings verified in simulation; physical interrupt latency not measured
OLED initialization failures	I2C setup/retry diagnostics and correct circuit/firmware loading; display cache reduces transfer work	OLED later verified in simulation and shown in screenshot; no bus waveform capture
Rapid encoder lag	Removed simulator interrupt storm; sampled full quadrature at safe points	Rapid navigation verified in simulation; hardware debounce timing remains unmeasured
Wake at 34 C caused Error/alarm cycling	Wait for actual DHT minimum interval before periodic read; recheck epoch	I verified the fix; actual retry latency not measured

Trade-offs and alternatives

Cooperative simulation is practical for the verified circuit but conceals preemption-related faults; physical validation must remain separate. Polling DHT pulses is simple and bounded but sensitive to interruptions. Timer input capture would reduce timing dependence, at greater driver complexity. DMA is an option for hardware after validating support; it cannot be assumed equivalent in Wokwi.

A FIFO plus separate latest-value mailboxes costs additional queue storage but makes consumer ownership explicit. Logging can drop samples under overload while display/alarm retain current data. Priority inheritance helps serial ownership but cannot remove UART transmission cost. Hardware PWM and cached OLED writes reduce CPU work without replacing blocking-task discipline.

No CPU percentage, stack watermark, memory-usage final build figure, or maximum response bound is claimed. The 14 KiB heap is configured capacity, not measured headroom. Physical safety/electrical suitability and comprehensive peripheral fault injection remain outside the recorded verification evidence.

8. Conclusion

The application demonstrates decomposition of acquisition, presentation, input, activity, alarm, and logging into native FreeRTOS tasks. Queues transfer complete samples, persistent events coordinate activity, and mutexes establish ownership of shared resources. Pure decisions were validated by 34 executed unit tests; I confirmed ten functional behaviors and performed three deliberate fault experiments with normal operation restored.

The main engineering lesson is that scheduling and data validity must be reasoned about together. A deadline does not guarantee sensor readiness, a clean log does not prove synchronization, and a cooperative simulator does not establish preemptive hardware timing. The report records what was tested and what remains unknown rather than treating each tool's passing result as universal assurance.

Recommended next work

Measure hardware response latency and stack/heap headroom; validate DHT pulse capture under interrupts; calibrate the light channel; capture I2C/encoder waveforms; and expand repeatable integration evidence. These steps would turn the current functional prototype into a better characterized embedded system.

References and repository evidence

[1] Paul Rodolf P. Castor, BCA182 Laboratory Activity No. 1: Real-Time Multisensor Room Monitoring System, September 2026, requirements 57-59 and related parts.

[2] Project repository: <https://github.com/uninhabitableelbow/bca182-freertos-multisensor>. The repository contains the source, circuit, tests, and supporting documentation.

[3] docs/part-xii-priorities.md and part-xiii-modules.md: scheduling justification and module ownership. docs/part-xiv-unit-tests.md: executed host-test coverage.

[4] docs/part-xv-static-analysis.md: full interpreted findings table. docs/part-xvi-functional-verification.md and part-xvii-fault-experiments.md: recorded simulation observations and restoration.

[5] Wokwi Blue Pill and project configuration: docs.wokwi.com/parts/board-stm32-bluepill and docs.wokwi.com/vscode/project-config. PlatformIO unit testing and Cppcheck: docs.platformio.org. FreeRTOS kernel: github.com/FreeRTOS/FreeRTOS-Kernel. STM32CubeF1: github.com/STMicroelectronics/STM32CubeF1.

[6] Prior STM32/Wokwi compatibility investigation acknowledged in README.md: github.com/monxx-ie/BCA182-freetos-multisensor.